

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Assignment No.: 1

Student Name: EMANDI HARI CHARAN

Roll No.: A24126552080

Date:

1. TITLE

Implementation of Inheritance Types in JavaScript

2. AIM

To design, implement, and execute a comprehensive JavaScript application demonstrating the five primary types of Object-Oriented Inheritance: Single Inheritance, Multilevel Inheritance, Hierarchical Inheritance, Multiple Inheritance using Mixins, and Hybrid Inheritance.

3. OBJECTIVE

The objectives of this assignment are:

- To understand Object-Oriented Programming (OOP) paradigms and prototype chain linkage in modern JavaScript (ES6+).
- To implement Single Inheritance using the class and extends keywords.
- To construct Multilevel Inheritance spanning Grandparent, Parent, and Child classes.
- To implement Hierarchical Inheritance where multiple specialized classes inherit from a single general base class.
- To simulate Multiple Inheritance using Mixin composition and Object.assign() on the class prototype.
- To construct Hybrid Inheritance combining multilevel inheritance with class specialization.

4. THEORY

Inheritance is a fundamental mechanism of Object-Oriented Programming (OOP) wherein a newly created class (subclass/child) inherits properties and methods from an existing class (superclass/parent). This facilitates code reusability, modular architecture, and hierarchical modeling.

In modern JavaScript (ECMAScript 2015/ES6), inheritance is syntactic sugar over prototype-based inheritance. This assignment explores the following five inheritance architectures:

- 1. Single Inheritance: A derived class inherits directly from a single base class (e.g., class Dog extends Animal). The child inherits all methods defined in the parent prototype.
- 2. Multilevel Inheritance: Involves an unbroken chain of inheritance where a class extends a child class (e.g., Child extends Parent, and Parent extends Grandparent). Methods from all ancestral prototypes are resolved through the prototype chain.

- 3. Hierarchical Inheritance: Multiple derived subclasses independently inherit from a common parent class (e.g., Student extends Person, and Teacher extends Person).
- 4. Multiple Inheritance (via Mixins): Traditional multiple inheritance (extending two or more parent classes) is disallowed in JavaScript to prevent ambiguities such as the Diamond Problem. However, it is effectively implemented using Object Mixins: standalone object literals composed onto a class prototype using Object.assign(Class.prototype, Mixin1, Mixin2).
- 5. Hybrid Inheritance: A multifaceted architecture combining two or more inheritance styles (e.g., multilevel inheritance combined with specialization: Vehicle -> Car -> ElectricCar).

Execution Workflow:

Class Blueprint Definition → Prototype Chain Linkage / Mixin Composition → Instance Instantiation (new) → Runtime Method Lookup via [[Prototype]] → Execution & Output Logging

5. ALGORITHM / PROCEDURE

1. Initialize the script inheritance.js within the assignments/assignment-1 workspace.
2. Implement Single Inheritance: Declare base class Animal with eat(). Declare class Dog extending Animal with bark(). Instantiate Dog and execute dog.eat() and dog.bark().
3. Implement Multilevel Inheritance: Declare class Grandparent with house(). Declare class Parent extending Grandparent with car(). Declare class Child extending Parent with bike(). Instantiate Child and invoke methods across all 3 tiers.
4. Implement Hierarchical Inheritance: Declare class Person with walk(). Declare class Student extending Person with study(). Declare class Teacher extending Person with teach(). Instantiate both subclasses and invoke shared and distinct methods.
5. Implement Multiple Inheritance using Mixins: Define Swimming object with swim() and Flying object with fly(). Define class Bird with eat(). Merge mixins onto Bird.prototype using Object.assign(Bird.prototype, Swimming, Flying). Instantiate Bird and invoke eat(), swim(), and fly().
6. Implement Hybrid Inheritance: Declare base class Vehicle with start(). Declare Car extending Vehicle with drive(). Declare ElectricCar extending Car with charge(). Instantiate ElectricCar and invoke all inherited methods.
7. Execute the script using Node.js via node inheritance.js and verify console outputs.

6. PROGRAM

inheritance.js

```
// Assignment 1: Implement Inheritance Types using JavaScript

// 1. SINGLE INHERITANCE
class Animal {
  eat() {
    console.log("Animal can eat");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog can bark");
  }
}
```

```
    }  
}  
  
console.log("1. Single Inheritance");  
let dog = new Dog();  
dog.eat();  
dog.bark();  
  
// 2. MULTILEVEL INHERITANCE  
class Grandparent {  
    house() {  
        console.log("Grandparent has a house");  
    }  
}  
  
class Parent extends Grandparent {  
    car() {  
        console.log("Parent has a car");  
    }  
}  
  
class Child extends Parent {  
    bike() {  
        console.log("Child has a bike");  
    }  
}  
  
console.log("\n2. Multilevel Inheritance");  
let child = new Child();  
child.house();  
child.car();  
child.bike();  
  
// 3. HIERARCHICAL INHERITANCE  
class Person {  
    walk() {  
        console.log("Person can walk");  
    }  
}  
  
class Student extends Person {  
    study() {  
        console.log("Student can study");  
    }  
}  
  
class Teacher extends Person {  
    teach() {  
        console.log("Teacher can teach");  
    }  
}  
  
console.log("\n3. Hierarchical Inheritance");
```

```
let student = new Student();
student.walk();
student.study();

let teacher = new Teacher();
teacher.walk();
teacher.teach();

// 4. MULTIPLE INHERITANCE USING MIXINS
const Swimming = {
  swim() {
    console.log("Can swim");
  }
};

const Flying = {
  fly() {
    console.log("Can fly");
  }
};

class Bird {
  eat() {
    console.log("Bird can eat");
  }
}

Object.assign(Bird.prototype, Swimming, Flying);

console.log("\n4. Multiple Inheritance");
let bird = new Bird();
bird.eat();
bird.swim();
bird.fly();

// 5. HYBRID INHERITANCE
class Vehicle {
  start() {
    console.log("Vehicle can start");
  }
}

class Car extends Vehicle {
  drive() {
    console.log("Car can drive");
  }
}

class ElectricCar extends Car {
  charge() {
    console.log("Electric car can charge");
  }
}
```

```

}

console.log("\n5. Hybrid Inheritance");
let electricCar = new ElectricCar();
electricCar.start();
electricCar.drive();
electricCar.charge();

```

7. CODE EXPLANATION

Syntax / Pattern	Explanation
<code>class Animal { eat() { ... } }</code>	Defines an ES6 class blueprint with an instance method on <code>Animal.prototype</code> .
<code>class Dog extends Animal</code>	Establishes prototype linkage where <code>Dog.prototype</code> inherits from <code>Animal.prototype</code> .
<code>let dog = new Dog()</code>	Instantiates a new Dog object; <code>dog.__proto__</code> references <code>Dog.prototype</code> .
<code>class Child extends Parent</code>	Extends the prototype chain: <code>Child -> Parent -> Grandparent -> Object.prototype</code> .
<code>class Student extends Person</code>	Hierarchical derivation allowing Student to access <code>Person.prototype</code> methods.
<code>class Teacher extends Person</code>	Second hierarchical derivation sharing <code>Person</code> base methods independently.
<code>const Swimming = { swim() { ... } }</code>	Declares an object mixin containing modular behaviors.

<code>Object.assign(Bird.prototype, ...)</code>	Copies enumerable own properties from mixins onto Bird.prototype, achieving multiple inheritance.
<code>class ElectricCar extends Car</code>	Hybrid chain combining multilevel inheritance with specialized charging capability.

8. TEST CASES AND EXECUTION DATA

Test Case	Input Name	Test Input	Expected Result	Actual Result	Status
TC-01	Single Inheritance	<code>dog.eat(),</code> <code>dog.bark()</code>	Logs 'Animal can eat' and 'Dog can bark'	Both logged cleanly	Pass
TC-02	Multilevel Inheritance	<code>child.house(),</code> <code>car(), bike()</code>	Child object accesses methods from all 3 hierarchy levels	All 3 levels executed	Pass
TC-03	Hierarchical: Student	<code>student.walk(),</code> <code>student.study()</code>	Student inherits walk() from Person, calls study()	Logged accurately	Pass
TC-04	Hierarchical: Teacher	<code>teacher.walk(),</code> <code>teacher.teach()</code>	Teacher inherits walk() from Person, calls teach()	Logged accurately	Pass
TC-05	Multiple (Mixins)	<code>bird.eat(),</code> <code>swim(),</code> <code>fly()</code>	Bird calls class method and mixin methods seamlessly	All 3 methods called	Pass
TC-06	Hybrid Inheritance	<code>electricCar.start(),</code> <code>drive(), charge()</code>	Calls methods spanning Vehicle, Car, and ElectricCar	Executed in sequence	Pass

9. OUTPUT

Output 1: Execution of All JavaScript Inheritance Types via Node.js Terminal

```
PS C:\Users\Hari Charan\OneDrive\Desktop\fullstack\assignments> node inheritance.js
1. Single Inheritance
Animal can eat
Dog can bark

2. Multilevel Inheritance
Grandparent has a house
Parent has a car
Child has a bike

3. Hierarchical Inheritance
Person can walk
Student can study
Person can walk
Teacher can teach

4. Multiple Inheritance
Bird can eat
Can swim
Can fly

5. Hybrid Inheritance
Vehicle can start
Car can drive
Electric car can charge
❖ PS C:\Users\Hari Charan\OneDrive\Desktop\fullstack\assignments>
```

10. RESULT

Thus, all five types of JavaScript inheritance (Single, Multilevel, Hierarchical, Multiple via Mixins, and Hybrid Inheritance) were successfully implemented, executed, and verified across all test cases. Prototype chain resolution and mixin composition functioned accurately without runtime errors.